

Guía de Integración API para la Obtención del Precio Promedio USDT/VES en Venezuela

Sección I: Introducción al Desafío del Precio USDT/VES: Por Qué el Mercado P2P es la Métrica Clave

Subsección 1.1: El Contexto Venezolano: Un Mercado No Tradicional

Para desarrollar una aplicación que maneje el precio del Tether (USDT) en Bolívars (VES), es fundamental comprender la naturaleza única del mercado venezolano. A diferencia de los mercados de divisas globales, el intercambio entre USDT y VES no se centraliza en un mercado *Spot* (de contado) tradicional.

Debido a factores como la hiperinflación y los controles de capital históricos, el USDT se ha convertido en un vehículo principal para el resguardo de valor y la realización de pagos.¹ Esta dinámica ha provocado que el mercado principal para este par sea el *Peer-to-Peer* (P2P). Plataformas como Binance, Bybit y Remitano actúan como intermediarios, permitiendo a los usuarios venezolanos comprar y vender USDT directamente entre ellos usando métodos de pago locales, como transferencias bancarias.²

Por lo tanto, los precios que se observan en plataformas como Coinbase o Bybit, que pueden

mostrar una tasa de conversión de 1 USDT a aproximadamente 230-231 VES⁴, no refleja un libro de órdenes de *Spot*. En su lugar, son un reflejo de las tasas de conversión de la plataforma o, más comunes, un agregado de las tasas vigentes en estos mercados P2P. La causa (inestabilidad económica) genera el efecto (un mercado P2P descentralizado), y la API que su aplicación debe consumir debe reflejar esta realidad.

Subsección 1.2: La Distinción Crítica: API de Mercado Spot vs. API de Mercado P2P

Un error común para los desarrolladores que abordan este problema es buscar una API de *Spot*.

- **Mercado Spot:** Se define como un libro de órdenes centralizado (Order Book) gestionado por el *exchange*. Los compradores y vendedores operan contra el libro del *intercambio*. Las API para este mercado son claras y ofrecen *endpoints* como `GET /api/v3/ticker/price`⁸ o `GET /api/v3/avgPrice`⁹, que devuelven un precio único.
- **Mercado P2P:** Funciona más como un "tablón de anuncios".¹⁰ Los usuarios individuales publican "anuncios" de compra o venta a un precio que ellos fijan, especificando sus métodos de pago (ej. Banco de Venezuela, Bancaribe, Pago Móvil).² El *exchange* simplemente actúa como un custodio (escrow) para el criptoactivo (USDT) hasta que se confirma el pago fiduciario (VES).³

El desafío para el desarrollador es que la API de "precio promedio" que intuitivamente buscaría, como `GET /api/v3/avgPrice` de Binance⁹, es la ruta incorrecta. Como se demostrará en la Sección II, este *endpoint* es irrelevante porque no existe un par de *Spot* USDT/VES en los principales *exchanges*.¹¹ Ignorar esta distinción es el principal obstáculo para resolver la consulta.

Subsección 1.3: Redefiniendo el "Precio Promedio": Un Valor Calculado, No Obtenido

El "precio promedio" que la aplicación necesita no es un dato único que se "obtiene" mediante una llamada GET. Es un dato que debe *calcularse*.

El proceso técnico correcto que la aplicación debe implementar, preferiblemente en un *backend* propio, es el siguiente:

1. **Obtener datos:** Utilizar una API que devuelva una *lista* de los anuncios de activos P2P (por ejemplo, los 10-20 primeros anuncios de venta de USDT).
2. **Datos adicionales:** De cada anuncio en la respuesta JSON, extraiga el precio (ej. 231.50) y la cantidad disponible (ej. 500.00 USDT) o los límites de la transacción.
3. **Calcular Promedio:** Calcular un *promedio* ponderado basado en el volumen o la liquidez de cada anuncio. Un promedio simple (suma de precios / número de anuncios) sería inexacto.

El objetivo técnico es implementar una lógica similar a la que se observa en⁴⁵, que calcula un promedio ponderado ($\$totalPrice / totalBought\$$). Este enfoque refleja con mucha más precisión el "precio de mercado real" que un usuario final realmente experimentaría al intentar comprar USDT.

Sección II: Evaluación de las API Oficiales de Exchanges (La Ruta Obvia y sus Obstáculos)

La primera ruta de investigación es consultar las API de los *exchanges* más grandes que operan en Venezuela. Sin embargo, esta ruta presenta barreras significativas.

Subsección 2.1: El Caso de Binance: El Mayor Mercado P2P, la API Incorrecta

Binance es, sin duda, la plataforma con el mayor volumen P2P en Venezuela.¹ Obtener sus datos sería ideal, pero existen dos problemas fundamentales.

- Análisis de la API Spot (GET /api/v3/avgPrice):
La API de Binance Spot es robusta y está bien documentada.¹⁵ El punto final GET /api/v3/avgPrice parece perfecto, ya que su descripción oficial es "Precio promedio actual de un símbolo".⁸
 - **Refutación (El Obstáculo Central):** Este *punto final* es inútil para este caso de uso. Requiere un parámetro symbol (ej. BNBBTC BTCUSDT). Sin embargo, el *símbolo*

USDTVES **no existe** en el mercado *Spot* de Binance. La documentación de la API Spot¹¹ especifica que la lista completa de símbolos de *trading* se obtiene del *endpoint* GET /api/v3/exchangeInfo . Al consultar este *endpoint* , o al revisar las listas de pares de *trading* de Binance¹², se confirma que USDTVESno es un par de *trading Spot* soportado.

- **Análisis de la API P2P (Inexistencia de un Endpoint Público):**
Dado que el mercado relevante es el P2P, la solución lógica sería la API P2P de Binance.
 - **Refutación (El Vacío de la API):** Binance no ofrece una API pública y oficial para leer los anuncios del mercado P2P. Una revisión de la documentación oficial de su API¹⁸ muestra que ofrecen API para Spot, Margin, Futures y Options, pero no se menciona una API de lectura pública para datos P2P. La ausencia de esta API es lo que ha llevado a la comunidad de desarrolladores a intentar métodos de *scraping* (extracción de datos web), como se insinúa en discusiones técnicas.¹⁹, lo cual es una práctica frágil y no recomendada para una aplicación en producción.

Subsección 2.2: El Caso de Bybit: La "P2P Open API" y su Barrera de Entrada

Bybit es otra plataforma con un mercado P2P USDT/VES activo.²A diferencia de Binance, Bybit sí publicita una "API abierta P2P".²¹

- **Refutación (El Muro del "Anunciante"):** Esta API, aunque existe, no es accesible para el público general de desarrolladores. El análisis de su documentación introductoria.²¹ revela un requisito crítico: "Para usar la API P2P, primero debe convertirse en *anunciante P2P* " .
 - La API está diseñada para que los *anunciantes* (comerciantes P2P) *gestionen* sus propias operaciones: crear anuncios, editar anuncios, ver sus órdenes.²¹No está diseñado para que un tercero consulte el libro de anuncios públicos. Esto la hace inviable para la aplicación del usuario.

Subsección 2.3: El Caso de Remitano: La Solución Oficial Más Viable

Remitano es otra plataforma establecida que opera un mercado P2P en Venezuela y soporta perfectamente el par USDT/VES.²³

- **Análisis de la API (v1 y v2):** Remitano proporciona una documentación para desarrolladores.²⁵
- **El Endpoint Correcto:** A diferencia de Binance y Bybit, la API de Remitano *sí* parece ofrecer los *endpoints* necesarios para consultar las ofertas P2P públicas. La documentación de la API v2²⁸ y los ejemplos²⁹ *puntos finales* de referencia como:
 - GET /v2/offers: Para obtener una lista de ofertas.
 - POST /v2/offers/best(Visto en un ejemplo de PHP)²⁹) Este *endpoint* parece permitir una consulta de la mejor oferta especificando `country_code`, `offer_type`(compra/venta) y `coin_currency`(USDT).
- **Autenticación:** El acceso a esta API requiere autenticación mediante una ACCESS_KEY y SECRET_KEY ²⁵, lo cual es un procedimiento estándar para proteger el servicio.

Conclusión: Remitano es la *única* plataforma P2P principal que parece ofrecer una API oficial, documentada y accesible para consultar las ofertas P2P, que es el *proxy* necesario para calcular el precio promedio.

Sección III: Evaluación de Soluciones API de Terceros (La Ruta No Oficial pero Pragmática)

Dada la falta de una API P2P pública y oficial por parte de Binance (el líder del mercado), ha surgido un ecosistema de servicios de terceros que cubren esta demanda.

Subsección 3.1: “Binance P2P API en zigzag” (vía RapidAPI)

Una solución destacada de terceros es la "Binance P2P API by zigzagway", alojada en el *mercado* de API RapidAPI.³¹

- **Función:** Este servicio actúa como un *scraper* (extractor de datos) que monitorea la página P2P pública de Binance y exponen los datos a través de una API RESTful limpia. Su descripción, "proporciona acceso directo a los anuncios de COMPRA y VENTA en vivo del mercado P2P de Binance"³¹, es precisamente la funcionalidad que se requiere.
- **Autenticación:** El consumo de esta API no requiere una clave de Binance, sino una clave

de RapidAPI (X-RapidAPI-Key) que debe incluirse en los *encabezados* de la solicitud.³²

- **Puntos finales:** Aunque los documentos³¹ y³¹ no pude cargar los detalles, la descripción³¹ implica que la API proporciona *endpoints* para anuncios de compra y venta, con filtros por criptomoneda (USDT), moneda fiduciaria (VES) y método de obtención de pago.

Subsección 3.2: Análisis de Costos, Límites y Riesgos (RapidAPI)

Esta solución, aunque funcionalmente ideal, introduce una serie de riesgos y costos operativos.

- **Costos y Límites:** RapidAPI opera con un modelo *freemium* .
 - Habrá un plan "BASIC" (gratuito) que permite un número limitado de solicitudes. La documentación general de RapidAPI indica que los aviones gratuitos pueden estar limitados a 1.000 solicitudes por hora y 500.000 por mes.³⁴
 - Si la aplicación del usuario escala más allá de estos límites, se requerirá una suscripción a un plan de pago.³⁴
- **Riesgos (Fragilidad):** Este es el inconveniente más significativo. El uso de esta API crea una cadena de dependencia frágil:
Su Aplicación -> RapidAPI -> API de zigzagway -> Página Web Pública de Binance
 - Esta API no es oficial. Si Binance decide cambiar el diseño HTML/CSS de su página P2P (lo cual pueden hacer en cualquier momento y sin previo aviso), el *scraper* de zigzagway fallará. Esto, a su vez, hará que la API de RapidAPI falle, y la aplicación del usuario dejará de funcionar. Este es un riesgo de fragilidad operativa muy alto para una aplicación en producción.

Sección IV: Estrategia Recomendada y Tabla Comparativa

Subsección 4.1: Justificación de la Estrategia (Enfoque Híbrido)

La selección de una API depende del equilibrio entre la precisión de los datos (volumen de mercado) y la robustez del sistema.

- **Recomendación Primaria (Producción):** Utilizar la **API oficial de Remitano** (ej. GET /v2/offerso POST /v2/offers/best).²⁸ Esta es la solución más robusta y profesional. Al ser una API oficial, está diseñada para el consumo de máquinas, está documentada²⁵ y es menos probable que falle sin previo aviso.
- **Recomendación Secundaria (Datos de Alto Volumen):** Utilizar la **API de RapidAPI ("zigzagway")** ³¹ *solo si es imperativo* tener el precio de Binance, que es el mercado de mayor volumen. El desarrollador debe aceptar los riesgos de fragilidad y los potenciales costos de escalada de RapidAPI.³⁴
- **Recomendación Arquitectónica (Crítica):** Independientemente de la API elegida, nunca debe llamarse a estas API directamente desde la aplicación cliente (la app móvil o web). Hacerlo expondría las claves de API (API Keys) y agotaría rápidamente los límites de tasa (rate limits).³⁶

La arquitectura correcta es construir un backend propio (un microservicio) que actúa como intermediario. Este backend debe:

1. Llamar a la API de Remitano o RapidAPI de forma segura desde el servidor.
2. Parsear la respuesta JSON y calcular el promedio ponderado.⁴⁵
3. **Almacenar en caché (cache)** este valor (ej. en Redis o en memoria) por un período corto (ej. 1 a 5 minutos).
4. Servir este valor *cacheado* a todos los clientes de la aplicación. Esto reduce masivamente las llamadas a la API externa, respeta los límites de tasa y mejora la velocidad de respuesta de la *aplicación*.

Subsección 4.2: Tabla Comparativa de Opciones de API para USDT/VES

Esta tabla resume el análisis de las Secciones II y III, proporcionando una guía de decisión clara.

Insumos de API	Punto final	Tipo de Dato	Soporte USDTVE	Requiere autentic	Costo (Nivel	Viabilidad para el
----------------	-------------	--------------	----------------	-------------------	--------------	--------------------

	relevante		S	acción.	Básico)	padre
Binance (Oficial)	GET /api/v3/avgPrice	Lugar	No ¹¹	Clave API	Gratis	Inviabile (Trampa Común)
Bybit (Oficial)	P2P Open API	P2P	Si ²	Clave API ²¹	Gratis	Inviabile (Requiere ser Anunciante) ²¹
Remitano (Oficial)	GET /v2/offers	P2P	Si ²⁵	Clave API ²⁵	Gratis	Alta (Recomendación Primaria)
RapidAPI (zigzagway)	(Endpoint de anuncios)	P2P (Binance)	Si ³¹	Clave API (RapidAPI)	Freemium ³⁴	Media (Frágil pero con datos de Binance)

Sección V: Guía de Implementación Técnica (Backend con Python)

A continuación, se presenta un ejemplo conceptual de cómo construir el *backend* de caché en Python, siguiendo la estrategia recomendada.

Paso 1: Instalar la Biblioteca Necesaria

Se utilizará la biblioteca `requests` para realizar las llamadas HTTP.

Intento

```
pip install requests
```

38

Paso 2: Consumir la API (Ejemplo con Remitano)

Este código muestra cómo estructurar la llamada a la API de Remitano, incluyendo la autenticación y los parámetros para la consulta P2P.

Pitón

```
# Ejemplo conceptual basado en [40, 46] y
```

```
import requests
```

```
# NOTA: La autenticación real de Remitano puede requerir firma HMAC.
```

```
# Este es un ejemplo simplificado usando Bearer token.
```

```
# Consulte la documentación de Remitano [30] para la implementación exacta.
```

```
API_URL = "https://api.remitano.com/api/v2/offers/best"
```

```
AUTH_HEADERS = {"Authorization": "Bearer SU_ACCESS_KEY"} # [40]: Headers personalizados
```

```
# Consultar por las mejores ofertas de VENTA (precio al que podemos COMPRAR USDT)
```

```
payload = {
```

```
    "country_code": "VE",
```

```
    "offer_type": "sell", # La gente está vendiendo USDT
```

```
    "coin_currency": "USDT",
```

```

    "fiat_currency": "VES" # Mapeo de
}

try:
    # Usar POST según el ejemplo de
    response = requests.post(API_URL, headers=AUTH_HEADERS, json=payload)

    # Lanzar una excepción si la respuesta es un error 4xx o 5xx
    response.raise_for_status()

    # Parsear la respuesta JSON [40]
    data = response.json()

    #... (Proceder al Paso 3 con 'data')

except requests.exceptions.RequestException as e:
    print(f"Error al contactar la API de Remitano: {e}")

```

Paso 3: Analizar el JSON y Calcular el Promedio Ponderado

Este es el paso crítico. Una vez obtenidos los datos, se debe calcular el promedio ponderado.

Pitón

```

# Lógica adaptada del concepto en

def calculate_weighted_average(offers_data):
    """
    Calcula el precio promedio ponderado de una lista de ofertas P2P.
    """
    total_fiat_value = 0.0
    total_crypto_volume = 0.0

    # Asumimos que 'data' tiene una clave 'offers' que es una lista

```

```
# Limitar a las primeras 10 ofertas para obtener un "promedio de mercado" relevante
offers_list = offers_data.get('offers',)[:10]
```

```
if not offers_list:
    return 0.0
```

```
for offer in offers_list:
    try:
        # Extraer precio y cantidad
        price = float(offer.get('price')) # ej: 231.50 (VES por USDT)
```

```
        # Usar la cantidad máxima disponible en la oferta como ponderación
        available_usdt = float(offer.get('max_coin_amount'))
```

```
        if available_usdt > 0:
            total_fiat_value += price * available_usdt
            total_crypto_volume += available_usdt
```

```
    except (TypeError, ValueError):
        # Omitir la oferta si los datos no son numéricos
        continue
```

```
if total_crypto_volume == 0:
    return 0.0
```

```
# El promedio ponderado es el valor total en fiat dividido por el volumen total de crypto
weighted_average_price = total_fiat_value / total_crypto_volume
```

```
return weighted_average_price
```

```
#... (Dentro del bloque 'try' del Paso 2)
# average_price = calculate_weighted_average(data)
# print(f"El precio promedio ponderado de USDT/VES es: {average_price:.2f} VES")
# (Aquí es donde se implementaría el CACHING)
```

Sección VI: Guía de Implementación Técnica (Frontend o Backend con JavaScript)

A continuación, se presenta un ejemplo conceptual usando JavaScript (Node.js o *frontend*) para consumir la API de terceros de RapidAPI.

Paso 1: Usar la API fetch con async/await

Se utilizará la API fetch, que es el estándar moderno en JavaScript para realizar aplicaciones de red, junto con la sintaxis async/await para un manejo asíncrono limpio.³³

Paso 2: Consumir la API (Ejemplo con RapidAPI "zigzagway")

Este código muestra cómo configurar los encabezados específicos de RapidAPI y manejar la respuesta.

JavaScript

```
// Ejemplo conceptual basado en y
```

```
// URL hipotética de la API en RapidAPI
```

```
const API_URL = 'https://binance-p2p-api.p.rapidapi.com/getAds';
```

```
const RAPIDAPI_KEY = 'SU_RAPIDAPI_KEY_PERSONAL';
```

```
const RAPIDAPI_HOST = 'binance-p2p-api.p.rapidapi.com';
```

```
async function getBinanceP2PPrice() {
```

```
  // Configuración de la solicitud, incluyendo headers personalizados [33]
```

```
  const fetchOptions = {
```

```
    method: 'POST', // O 'GET', dependiendo de la API específica
```

```
    headers: {
```

```
      'X-RapidAPI-Key': RAPIDAPI_KEY,
```

```

    'X-RapidAPI-Host': RAPIDAPI_HOST,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    asset: 'USDT',
    fiat: 'VES',
    tradeType: 'SELL', // Anuncios de "SELL" (precio al que el usuario compra)
    page: 1
  })
};

try {
  const response = await fetch(API_URL, fetchOptions);

  // Verificar si la respuesta es exitosa (status 200-299) [33]
  if (!response.ok) {
    throw new Error(`Error HTTP: ${response.status}`);
  }

  // Parsear la respuesta JSON [41, 42]
  const data = await response.json();

  //... (Proceder al Paso 3 con 'data')

} catch (error) {
  // Manejo de errores de red o de parseo JSON [43]
  console.error("Error al obtener datos P2P de RapidAPI:", error.message);
}
}

```

Paso 3: Analizar el JSON y Calcular el Promedio Ponderado

La lógica de cálculo es idéntica a la de Python, adaptada a la sintaxis de JavaScript.

JavaScript

// Lógica adaptada del concepto en

```
function calculateWeightedAverage(adsData) {  
  let totalFiatValue = 0.0;  
  let totalCryptoVolume = 0.0;  
  
  // Asumimos que 'adsData' tiene una clave 'data' que es una lista de anuncios  
  const adsList = adsData.data? adsData.data.slice(0, 10) ::  
  
  if (adsList.length === 0) {  
    return 0.0;  
  }  
  
  for (const ad of adsList) {  
    // La estructura exacta (ej. 'adv.price') dependerá de la respuesta de la API  
    const price = parseFloat(ad.adv.price);  
    const availableUsdt = parseFloat(ad.adv.tradableQuantity); // Cantidad disponible  
  
    if (!isNaN(price) &&!isNaN(availableUsdt) && availableUsdt > 0) {  
      totalFiatValue += price * availableUsdt;  
      totalCryptoVolume += availableUsdt;  
    }  
  }  
  
  if (totalCryptoVolume === 0) {  
    return 0.0;  
  }  
  
  const weightedAveragePrice = totalFiatValue / totalCryptoVolume;  
  return weightedAveragePrice;  
}  
  
//... (Dentro del bloque 'try' del Paso 2)  
// const averagePrice = calculateWeightedAverage(data);  
// console.log(`El precio promedio ponderado de USDT/VES es: ${averagePrice.toFixed(2)} VES`);  
// (Aquí es donde se implementaría el CACHING si esto fuera un backend Node.js)
```

Sección VII: Consideraciones Avanzadas y

Conclusión Estratégica

Subsección 7.1: La importancia crítica del almacenamiento en caché

Es fundamental reiterar la recomendación arquitectónica de la Sección 4.1. Llamar a una API externa en cada carga de la aplicación o por cada usuario es una práctica insostenible.

- **Límites de Tasa:** Todas las API, ya sean gratuitas o de pago, imponen límites de tasa (Rate Limits) para prevenir abusos.³⁶Un plan gratuito de RapidAPI puede limitar a 1000 solicitudes por hora.³⁴Si 1000 usuarios abren la aplicación en una hora, el límite se agota.
- **Rendimiento:** Las llamadas a API externas son lentas.
- **Solución:** La implementación de un *backend* propio que almacena en caché el precio calculado (ej. por 1 minuto) asegura que miles de usuarios puedan obtener el precio casi instantáneamente, mientras que el *backend* solo realiza una llamada a la API externa por minuto.

Subsección 7.2: Estrategias de Resiliencia y Resiliencia

Una aplicación robusta en producción no debe depender de un único punto de falla. Dado que el precio del USDT/VES es crítico para la funcionalidad de la aplicación, se debe diseñar una estrategia de *respaldo* (respaldo).

El *backend* debería programarse para ser resiliente:

1. **Intento 1:** Intentar obtener el precio de la API primaria (ej. Remitano²⁹).
2. **Intento 2 (En caso de fallo):** Si la API de Remitano falla (error 500, *timeout*), el *backend* debe automáticamente intentar obtener el precio de la API secundaria (ej. RapidAPI³¹).
3. **Intento 3 (En caso de fallo total):** Si ambas API fallan, el *backend* no debe devolver un error. Debe servir el último valor exitoso almacenado en la caché (un valor "rancio"), quizás con una marca de tiempo para indicar que el precio no está actualizado en tiempo real.

Subsección 7.3: Conclusión Final

La tarea de obtener el "precio promedio" del USDT/VES no es un simple consumo de API, sino un problema de ingeniería de datos que implica obtener, limpiar, calcular y almacenar en caché.

El error más común es buscar una API de *Spot* (como GET /api/v3/avgPrice de Binance ⁹), que es irrelevante para el mercado P2P venezolano.

La solución más profesional y estable es utilizar la **API oficial de Remitano** ²⁹, que proporciona *endpoints* documentados para consultar ofertas P2P. La alternativa, que utiliza los datos de Binance (el mercado de mayor volumen), es usar un *wrapper* de terceros como el que se encuentra en **RapidAPI** ³¹, aceptando los riesgos de fragilidad que esto conlleva. En ambos casos, la implementación de un *backend* de caché intermedio es una necesidad de arquitectura para la escalabilidad y la robustez.

Fuentes citadas

1. Binance P2P Trading in Venezuela: A New Era for Cryptocurrencies, acceso: noviembre 12, 2025, <https://www.binance.com/en/square/post/12215816198898>
2. Buy USDT with VES via Bybit P2P Crypto Trading, acceso: noviembre 12, 2025, <https://www.bybit.com/en/p2p/buy/USDT/VES>
3. Buy and Sell Tether (USDT) with Bank Transfer (Venezuela) | Binance P2P, acceso: noviembre 12, 2025, <https://p2p.binance.com/en/trade/bankvenezuela>
4. Convert Tether (USDT) to Venezuelan Bolívar Soberano (VES) - Coinbase, acceso: noviembre 12, 2025, <https://www.coinbase.com/converter/usdt/ves>
5. Intercambiar Tether USDT a Venezuelan Bolívar Soberano VES - Coinbase, acceso: noviembre 12, 2025, <https://www.coinbase.com/es-la/converter/usdt/ves>
6. Tasa de cambio USDT/VES | Convertir Tether a Bolívar venezolano - Bybit, acceso: noviembre 12, 2025, <https://www.bybit.com/es-MX/convert/usdt-to-ves/>
7. intercambiar Tether (USDT) a Bolívar soberano (VES) | Coinbase: usdt a bolivares, usdt precio hoy, comprar usdt a bcv, acceso: noviembre 12, 2025, <https://www.coinbase.com/es-us/converter/usdt/ves>
8. Binance.US API Documentation: Introduction, acceso: noviembre 12, 2025, <https://docs.binance.us/>
9. Market Data endpoints | Binance Open Platform, acceso: noviembre 12, 2025, <https://developers.binance.com/docs/binance-spot-api-docs/rest-api/market-data-endpoints>
10. Bitfinex Launches Peer-to-Peer Platform 'Bitfinex P2P' in Venezuela, Argentina

- and Colombia, acceso: noviembre 12, 2025, <https://blog.bitfinex.com/media-releases/bitfinex-launches-peer-to-peer-platform-bitfinex-p2p-in-venezuela-argentina-and-colombia/>
11. How to Retrieve Binance Spot Symbols via API?, acceso: noviembre 12, 2025, <https://academy.binance.com/lt/articles/how-to-retrieve-binance-spot-symbols-via-api>
 12. Trade Limits | Binance.US, acceso: noviembre 12, 2025, <https://www.binance.us/trade-limits>
 13. What is the Best crypto exchange with spot USDT pairs? : r/JapanFinance - Reddit, acceso: noviembre 12, 2025, https://www.reddit.com/r/JapanFinance/comments/1fhiobf/what_is_the_best_crypto_exchange_with_spot_usdt/
 14. Binance P2P: Buy and Sell Tether (USDT) with Your Preferred Payment Methods, acceso: noviembre 12, 2025, <https://p2p.binance.com/en>
 15. General API Information | Binance Open Platform, acceso: noviembre 12, 2025, <https://developers.binance.com/docs/binance-spot-api-docs/rest-api>
 16. How to retrieve a list of all market pairs like ETH/BTC using Binance API? - Stack Overflow, acceso: noviembre 12, 2025, <https://stackoverflow.com/questions/55549499/how-to-retrieve-a-list-of-all-market-pairs-like-eth-btc-using-binance-api>
 17. Listings on Binance.US | Supported crypto, networks, and trading pairs, acceso: noviembre 12, 2025, <https://support.binance.us/en/articles/9842915-listings-on-binance-us-supported-crypto-networks-and-trading-pairs>
 18. Binance APIs, acceso: noviembre 12, 2025, <https://www.binance.com/en/binance-api>
 19. API binance P2P I only access a part (only the BUY) and not all of it (BUY and SELL), acceso: noviembre 12, 2025, <https://stackoverflow.com/questions/67793326/api-binance-p2p-i-only-access-a-part-only-the-buy-and-not-all-of-it-buy-and-s>
 20. Sell USDT to VES via Bybit P2P Crypto Trading, acceso: noviembre 12, 2025, <https://www.bybit.com/en/p2p/sell/USDT/VES>
 21. Introduction to P2P Open API - Bybit, acceso: noviembre 12, 2025, <https://www.bybit.com/en/help-center/article/Introduction-to-P2P-Open-API>
 22. bybit-exchange/bybit_p2p: Official Bybit P2P API integration library written in Python. - GitHub, acceso: noviembre 12, 2025, https://github.com/bybit-exchange/bybit_p2p
 23. Compre y venda USDT Tether USDT con VES undefined en Venezuela con tarifa 0 | Intercambio P2P | remitano, acceso: noviembre 12, 2025, <https://remitano.com/ve/p2p/usdt>
 24. Remitano: Buy, Sell & Convert Bitcoin, Ethereum & Crypto with Fiat, USD or Gift Card, acceso: noviembre 12, 2025, <https://remitano.com/>
 25. Getting Started | Remitano Developers, acceso: noviembre 12, 2025, <https://developers.remitano.com/docs/getting-started/getting-started/>

26. Sell USDT Tether USDT with VES undefined in Venezuela with 0 Fee | P2P Exchange | Remitano, acceso: noviembre 12, 2025, <https://remitano.com/usdt/ve-en/sell>
27. API Explorer - Remitano Developers, acceso: noviembre 12, 2025, <https://developers.remitano.com/api-explorer/>
28. API Explorer - Remitano Developers, acceso: noviembre 12, 2025, <http://developers.remitano.com/api-v2-explorer/>
29. Remitano PHP API Client - GitHub, acceso: noviembre 12, 2025, <https://github.com/remitano/remitano-php>
30. Access Key - Remitano Developers, acceso: noviembre 12, 2025, <https://developers.remitano.com/docs/getting-started/access-key/>
31. Binance P2P API - Rapid API, acceso: noviembre 12, 2025, <https://rapidapi.com/zigzagway/api/binance-p2p-api>
32. How to Use the Python Requests Module With REST APIs - Nylas, acceso: noviembre 12, 2025, <https://www.nylas.com/blog/use-python-requests-module-rest-apis/>
33. Using the Fetch API - MDN Web Docs, acceso: noviembre 12, 2025, https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
34. Monetizing Your API on rapidapi.com, acceso: noviembre 12, 2025, <https://docs.rapidapi.com/docs/monetizing-your-api-on-rapidapicom>
35. FAQs - rapidapi.com API Hub, acceso: noviembre 12, 2025, <https://docs.rapidapi.com/docs/faqs>
36. CoinMarketCap API Pricing Plans, acceso: noviembre 12, 2025, <https://pro.coinmarketcap.com/api/pricing>
37. How Do You Handle API Rate Limits When Calling a Crypto Data Endpoint? - Token Metrics, acceso: noviembre 12, 2025, https://www.tokenmetrics.com/blog/mastering-api-rate-limits-crypto-data-integration?74e29fd5_page=121
38. Consuming APIs with Python - Marcos - Medium, acceso: noviembre 12, 2025, <https://mmarcosab.medium.com/consuming-rest-apis-with-python-b32e9f36afa4>
39. Python and REST APIs: Interacting With Web Services, acceso: noviembre 12, 2025, <https://realpython.com/api-integration-in-python/>
40. How do I fetch JSON using JavaScript Fetch API? - ReqBin, acceso: noviembre 12, 2025, <https://reqbin.com/code/javascript/wc3qbk0b/javascript-fetch-json-example>
41. API Documentation | Finnhub - Free APIs for realtime stock, forex, and cryptocurrency. Company fundamentals, economic data, and alternative data., acceso: noviembre 12, 2025, <https://finnhub.io/docs/api>
42. Calculate cryptocurrency average price from Binance API - GitHub, acceso: noviembre 12, 2025, <https://gist.github.com/luan0ap/58ad674a8bb734a9d44fecf71dac7c8d>